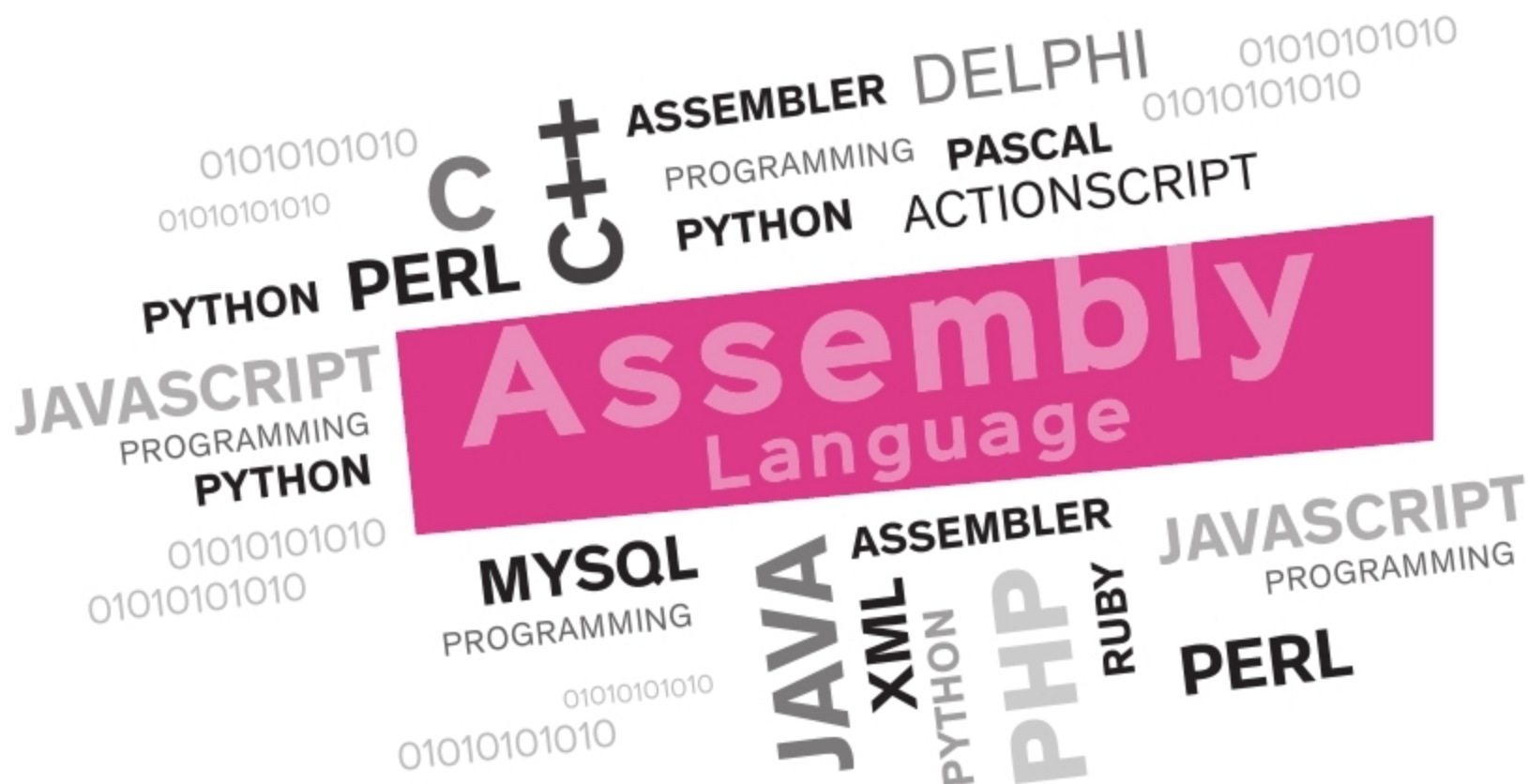


Assembly Language Programming in Linux: An Overview

One often comes across a lot of ads encouraging children to learn programming, which can be a very useful skill to have these days. This article focuses on assembly language programming in Linux, which figures at number 17 on the TIOBE index.



If you plan to make programming your career, it is a good idea to learn a programming language like assembly language at some stage. There are a number of reasons for this. First, it is still popular and ranks among the top 20 in the TIOBE index. The Stack Overflow Developer Survey 2020 ranked the assembly language 17th among the most wanted programming languages. (But I do not want to misrepresent facts, as assembly language is also featured in the list of the most dreaded languages — in fact, it's ranked 4th here. This language is also notorious for its steep learning curve.)

Second, it is widely used in the industry. Assembly language is used while designing operating systems and compilers. Of course, nowadays, a lot of this code is written in high-level languages like C, C++, etc. But certain parts of the kernel of the operating system are still written in assembly language. For example, while writing device drivers for a machine, programmers often rely on assembly coding. Assembly programming is also used in certain phases of compilation. For example, code optimisation phase of compilers often involve assembly coding. Essentially, when

you are programming on 'bare metal' (when you are accessing the hardware directly), you still need some assembly coding, whatever the application may be. Assembly language code can run very fast as there is one less translation to machine code when compared with high-level languages. In the earlier days, when resources like memory and processor speed were limited, this was a great advantage. Nowadays, even laptops have gigabytes of RAM and processors have gigahertz of speed. Does it make assembly language programming less attractive?

